CS3212 Algorithms

Aya Zirikly
10/27/2026

Graph Foundations & Traversals

```
Sample Graph (4 Vertices: 0, 1, 2, 3):

(0) ------- (1)
 |         / |
 |       /   |
 |     /     |
(2) ------- (3)

Adjacency List Representation (Array of Lists):

[0] -> [1] -> [2]
[1] -> [0] -> [2] -> [3]
[2] -> [0] -> [1] -> [3]
[3] -> [1] -> [2]
```

```
Sample Graph (4 Vertices: 0, 1, 2, 3):

(0) ------- (1)
 |         / |
 |       /   |
 |     /     |
(2) ------- (3)

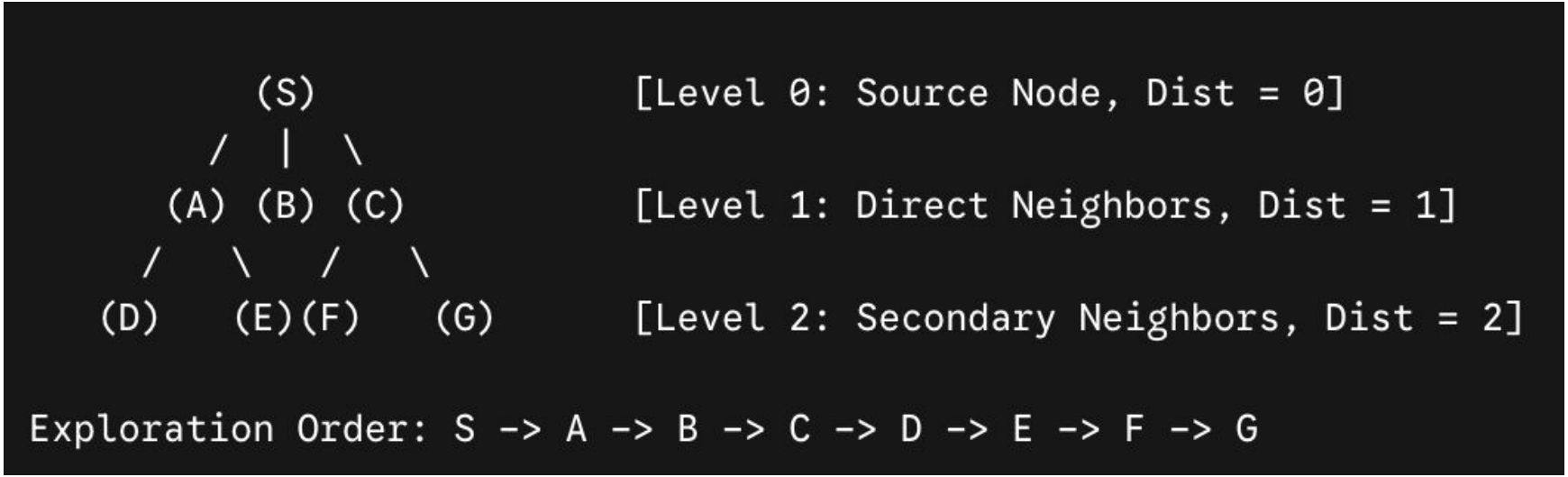
Adjacency Matrix Representation (4 x 4 Grid):

      0   1   2   3
    +---------------
  0 | 0   1   1   0
  1 | 1   0   1   1
  2 | 1   1   0   1
  3 | 0   1   1   0
```

Introduction to Breadth-First Search (BFS)

- **Level-Order Traversal:** BFS explores a graph level-by-level, radiating outward from a starting source node.
- **Exploration Strategy:** Visits all immediate neighbors at distance k before moving to any nodes at distance k + 1.
- **Shortest Path Property:** In unweighted graphs, the first time BFS reaches a target node, it has found the shortest path (fewest number of edges) from the source.
- **Core Data Structure:** Uses a First-In, First-Out (FIFO) Queue to maintain the boundary of nodes to explore.

Introduction to Breadth-First Search (BFS)



How BFS Works

Primary Auxiliary Structures

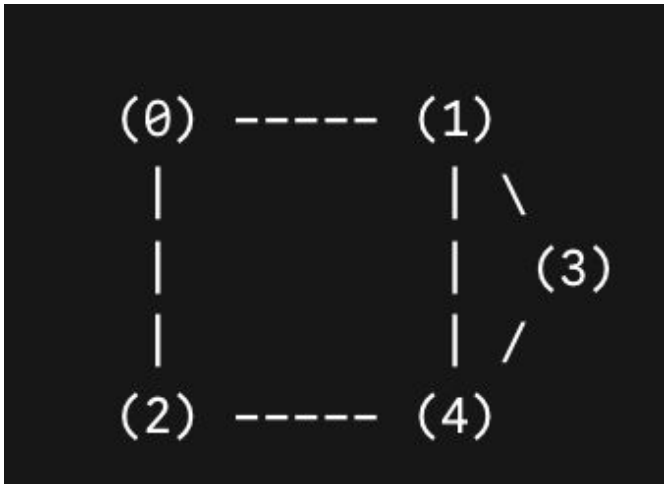
- **FIFO Queue:** Holds discovered nodes awaiting neighbor exploration.
- **Visited Array / Set:** Tracks visited vertices to prevent infinite loops in cyclic graphs and redundant processing.
- **Distance Array (`dist[]`):** Stores the shortest edge count from the source node to every other node (initialized to Infinity).
- **Parent Array (`parent[]`):** Records the predecessor of each vertex to reconstruct the exact path back to the source.

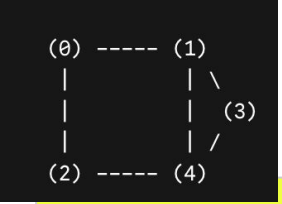
1. Mark `source` as visited, set `dist[source] = 0`, and add `source` to the queue.
2. While the queue is not empty:
    - Remove the front node `u`.
    - Iterate through all adjacent neighbors `v` of `u`.
    - For each unvisited neighbor `v`:
        - Mark `v` as visited.
        - Set `dist[v] = dist[u] + 1`.
        - Set `parent[v] = u`.
        - Add `v` to the queue.

```
BFS(G, source)
// Initialization
FOR EACH vertex v IN G.V:
    visited[v] = FALSE
    dist[v]    = ∞
    parent[v]  = NULL
visited[source] = TRUE
dist[source]    = 0
Q = empty queue
ENQUEUE(Q, source)

// Main Traversal Loop
WHILE Q is not empty:
    u = DEQUEUE(Q)
    FOR EACH neighbor v OF u:
        IF visited[v] is FALSE:
            visited[v] = TRUE
            dist[v]    = dist[u] + 1
            parent[v]  = u
            ENQUEUE(Q, v)
```

Step-by-Step Traversal Example





Processed Node	Queue State (Front -> Rear)	Visited Set	Distance Updates	Action / Notes
Start	[0]	{0}	dist[0] = 0	Initialize source node 0
0	[1, 2]	{0, 1, 2}	dist[1]=1, dist[2]=1	Dequeue 0; Enqueue unvisited neighbors 1, 2
1	[2, 3, 4]	{0, 1, 2, 3, 4}	dist[3]=2, dist[4]=2	Dequeue 1; Enqueue unvisited neighbors 3, 4
2	[3, 4]	{0, 1, 2, 3, 4}	None	Dequeue 2; Neighbor 4 is already visited
3	[4]	{0, 1, 2, 3, 4}	None	Dequeue 3; Neighbor 4 is already visited
4	[]	{0, 1, 2, 3, 4}	None	Dequeue 4; Queue empty. Search terminates

Path Reconstruction

Reconstructing the Shortest Path

- The `parent[]` array forms an inverted tree rooted at the source.
- To retrieve the path from `source` to `target`, backtrack from `target` to `source` using `parent[]` pointers, then reverse the sequence.

```
Reconstructing path to Target Node 3:

  Target (3) ---> parent[3] = 1 ---> parent[1] = 0 ---> parent[0] = NULL

  Backtracked Sequence: [3, 1, 0]
  Reversed Final Path:  0 -> 1 -> 3 (Length: 2 edges)
```

```
FUNCTION Reconstruct_Path(parent, source, target):
    path = []
    curr = target

    WHILE curr is not NULL:
        APPEND curr TO path
        curr = parent[curr]

    REVERSE(path)

    IF path[0] == source:
        RETURN path
    ELSE:
        RETURN "No path exists"
```

Introduction to Depth-First Search (DFS)

Deep-First Traversal: DFS explores a graph by diving as deep as possible along each branch before backtracking.

Exploration Strategy: From a starting vertex, DFS visits an unvisited neighbor and recursively continues along that path until it reaches a node with no unvisited neighbors (a dead end), then backtracks.

Data Structure: Uses a Last-In, First-Out (LIFO) Stack, either explicitly or implicitly via the system call stack (recursion).

How does it work?

CLRS 3-Coloring Model

To track state during exploration, the CLRS algorithm assigns each vertex a color:

- **WHITE:** Unvisited vertex.
- **GRAY:** Discovered vertex, currently being explored (on the call stack).
- **BLACK:** Finished vertex; all outgoing edges and reachable descendants have been fully explored.

CLRS Timestamps

DFS records two timestamps for each vertex `u`:

- `u.d` **(Discovery Time):** The step counter when `u` is first discovered (turns GRAY).
- `u.f` **(Finishing Time):** The step counter when `u` finishes exploration (turns BLACK).
- **Parent Pointer (`u.pi`):** Points to the vertex that discovered `u`.

DFS Pseudocode

```
DFS(G):
    FOR EACH vertex u IN G.V:
        u.color = WHITE
        u.pi = NIL

    time = 0

    FOR EACH vertex u IN G.V:
        IF u.color == WHITE:
            DFS-VISIT(G, u)


DFS-VISIT(G, u):
    time = time + 1
    u.d = time
    u.color = GRAY

    FOR EACH vertex v IN G.Adj[u]:
        IF v.color == WHITE:
            v.pi = u
            DFS-VISIT(G, v)

    u.color = BLACK
    time = time + 1
    u.f = time
```